
QUAID-E-AWAM UNIVERSITY OF ENGINEERING SCIENCE & TECHNOLOGY

Advanced Functions and Recursion

Week 07 – Day 01

Atta Muhammad

attapanhyar@quest.edu.pk

+923331971110

Quaid-e-Awam University of Engineering, Science and Technology

Review of Functions in Python

- **Overview**: Quick review of function definition, parameters, return values, and scope

Anonymous Functions (**lambda** Functions)

- **Introduction to Lambda**: Brief overview of lambda functions as anonymous, one-liner functions.

- **Syntax**: lambda arguments: expression

Using Lambda Functions in Python

Practical Uses: Common scenarios, like sorting, mapping, and filtering.

```
mynum = [1,2,3,4,5]
x = lambda x: [i**2 for i in  x if i%2==0]
print(x(mynum))
```

Hands-On Exercise: Sorting with Lambda

- **Task**: Write a lambda function to sort a list of dictionaries by age.

```
people = [{'name': 'Alice', 'age': 30}, {'name': 'Bob', 'age': 25}]
```

Advanced Lambda Examples

```
def myfunc(n):
  return lambda a : a * n
mydoubler = myfunc(2)
mytripler = myfunc(3)

print(mydoubler(11))
print(mytripler(11))
```

Functional Programming Overview

- Functional programming decomposes a problem into a set of functions.
- Ideally, functions only take inputs and produce outputs, and don't have any internal state that affects the output produced for a given input.
- **Concepts**: Introduction to map(), filter(), and reduce() functions in Python.
- **Why Functional Programming?**: Benefits like clean, readable, and concise code.

map() Function

- **Purpose**: Apply a function to each item in an iterable.
- The **map()** function is used to apply a given function to every item of an iterable, such as a list or tuple, and returns a map object (which is an iterator).

```
def square(x):
    return x*x
number = [1,2,3,4,5,6,7]
sqaureIterator = map(square, number)
print (list(sqaureIterator))
```

```
s = ['1', '2', '3', '4']
res = map(int, s)
print(list(res))
```

Lambda with map()

We can use a **lambda function** instead of a custom function with **map()** to make the code shorter and easier.

```
a = [1, 2, 3, 4]
# Using lambda function in "function" parameter
# to double each number in the list
res = list(map(lambda x: x * 2, a))
print(res)
```

we use **map()** to convert a list of temperatures from Celsius to Fahrenheit.

```
celsius = [0, 20, 37, 100]
fahrenheit = map(lambda c: (c * 9/5) + 32, celsius)
print(list(fahrenheit))
```

Advance Use of Map

filter() Function

- The **filter()** method filters the given sequence with the help of a function that tests each element in the sequence to be true or not.
- **Purpose**: Filter items in an iterable based on a condition.

```
numbers = [1, 2, 3, 4, 5]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
```

Filtering Letters

```
# function that filters vowels
def fun(variable):
    letters = ['a', 'e', 'i', 'o', 'u']
    if (variable in letters):
        return True
    else:
        return False

sequence = ['g', 'e', 'e', 'j', 'k', 's', 'p', 'r']

filtered = filter(fun, sequence)
print('The filtered letters are:')
for s in filtered:
    print(s)
```

Filter Function in Python with Lambda

```
# a list contains both even and odd numbers.
seq = [0, 1, 2, 3, 5, 8, 13]

# result contains odd numbers of the list
result = filter(lambda x: x % 2 != 0, seq)
print(list(result))

# result contains even numbers of the list
result = filter(lambda x: x % 2 == 0, seq)
print(list(result))
```

Filtering Objects

```python
# Example filtering objects based on attribute
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
people = [
    Person('Alice', 25),
    Person('Bob', 30),
    Person('Charlie', 22)
]
# Filter people older than 25
filtered_people = list(filter(lambda person: person.age > 25, people))
for person in filtered_people:
    print(person.name, person.age)
```

Introduction to Recursion

- **Definition**: A function that calls itself to solve a problem in smaller steps.
- **Basic Structure**:
  - Base case (stopping condition)
  - Recursive case

Example of a Recursive Function: Factorial Calculation

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)
print(factorial(5))
```

Understanding the Recursive Process: Fibonacci Sequence

Definition: $F(n) = F(n-1) + F(n-2)$

Recursion vs. Iteration

- **Comparison**: Pros and cons of recursion vs. iterative solutions.

- **Memory and Performance**: Recursion may lead to high memory usage and stack overflow.

Practical Tips for Using Recursion

- When to Use Recursion: Scenarios like tree traversal, directory navigation, and solving divide-and-conquer problems.
- Alternatives: Iteration